

There are many ways for an OS to schedule the time that threads enjoy the use of system resources. The simplest of the lot is the *round robin* method. Each thread is loaded turn by turn, and once the OS has reached the last thread, it returns to the first and starts all over again. Most OSes use *priority-based scheduling*, where higher priority threads are given bigger chunks of time. The priority of the thread is sometimes coded in by the programmer, but the operating system is the ultimate authority in this matter. It puts its own critical processes at a higher priority, and if it can give a thread the priority level it asks for, it does.

1.4 The Manager—Memory Management

The memory on your machine is divided into two categories. First, there is the system's physical memory—RAM chips. These chips are built for exchanging data with the CPU at very high speeds. Unfortunately, high speeds come at high costs, and even the richest know that it isn't wise to invest all one's money in RAM. Instead, we load our systems with all the physical memory our budgets allow, and add to it a cheaper (and slower) storage medium like a hard disk drive. This hard drive is now used to provide us with *virtual memory*.

Let us look at an OS running multiple tasks. One option for it would be to divide the available physical memory among the tasks, and give each portion to one task. This may make sense for two, maybe three processes which will get all the memory they need, but any more and your system would be slower than a pregnant hippo. Rather than face this, a better option would be to let each process have the memory it needs, and should the physical memory fall short, to store the process' data on a hard disk, and bring it into the physical memory when needed. This space that the operating system uses on the hard disk is called *virtual memory*. Using virtual memory, the OS 'fools' programs into believing that they have the system memory all to themselves.